ROS2上观看摄像头画面

ROS2上观看摄像头画面

- 0.在一个linux系统下安装docket
- 1.启动ROS-Wifi图传模块 ROS2代理
  - 发布节点
  - 话题信息
  - 消息类型
  - 话题发布频率
- 2.ROS2获取摄像头画面
- 3.在RVIZ里面查看图片
- 4.注意事项

0.在一个linux系统下安装docket

```
# **如果使用的是资料提供的系统此步跳过**
sudo apt update
sudo apt install docker.io docker-compose
docker -v
```

如图所示 docker就下载并安装完成

```
yahboom@yahboom-VM:~$ docker -v
Docker version 24.0.7, build afdd53b
```

1.启动ROS-Wifi图传模块 ROS2代理

1. 先把ROS-Wifi图传模块的STA模式启动(默认启动) 让esp32连接到所处环境的一个能上网的wifi
   注意：使用AP模式是不能进入代理的，因为进去代理需要网络，AP模式是没有网络
2. 代理主机也要和ROS-Wifi图传模块连接到同一个局域网
   代理主机：是一个装有ROS2(humble版本)的Linux系统
3. 查询代理主机的ip地址
   输入：ifconfig的命令

```
yahboom@yahboom-VM:~$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
        ether 02:42:93:f5:44:96  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.2.100  netmask 255.255.255.0  broadcast 192.168.2.255
        inet6 fe80::eeb4:aaad:2fb9:8c7  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:df:42:3c  txqueuelen 1000  (Ethernet)
        RX packets 1160  bytes 839024 (839.0 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 651  bytes 263777 (263.7 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

4. 然后使用串口工具，配置ROS-Wifi图传模块的ros2代理主机地址

- 比如上图，我的ip地址是**192.168.2.100**，那么串口的配置如图所示

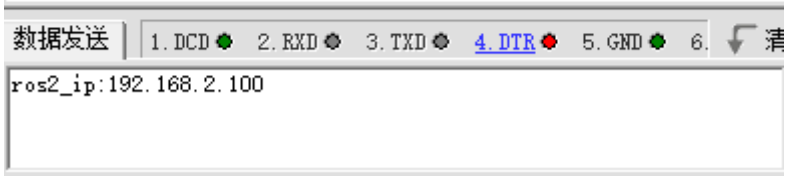


- 点击发送后，有两种信息打印，**无论那种都是配置完成的了**

1. 如果ROS-Wifi图传模块原本的代理ip和现在的不一样，就会更新现在的代理ip并重启

```
[2024-03-29 14:45:45.751]# RECV ASCII>
ESP-ROM:esp32s3-20210327
Build:Mar 27 2021
rst:0x3 (RTC_SW_SYS_RST),boot:0x8 (SPI_FAST_FLASH_BOOT)
Saved PC:0x40375a0c
SPIWP:0xee
mode:DIO, clock div:1
load:0x3fce3818,len:0x514
load:0x403c9700,len:0x4
load:0x403c9704,len:0xb58
load:0x403cc700,len:0x2a8c
entry 0x403c98fc

[2024-03-29 14:45:45.937]# RECV ASCII>
□[0;32mI (196) cpu_start: Multicore app□[0m
□[0;32mI (197) esp_psram: Found 2MB PSRAM device□[0m
□[0;32mI (197) esp_psram: Speed: 80MHz□[0m
□[0;32mI (198) cpu_start: Pro cpu up.□[0m
□[0;32mI (201) cpu_start: Starting app cpu, €
[2024-03-29 14:45:49.631]# RECV ASCII>
YAHBOOM VerSion:V2.0.5
```



2. 如果ROS-Wifi图传模块原本代理的ip就和现在一样，ROS-Wifi图传模块则会返回一个OK的信息



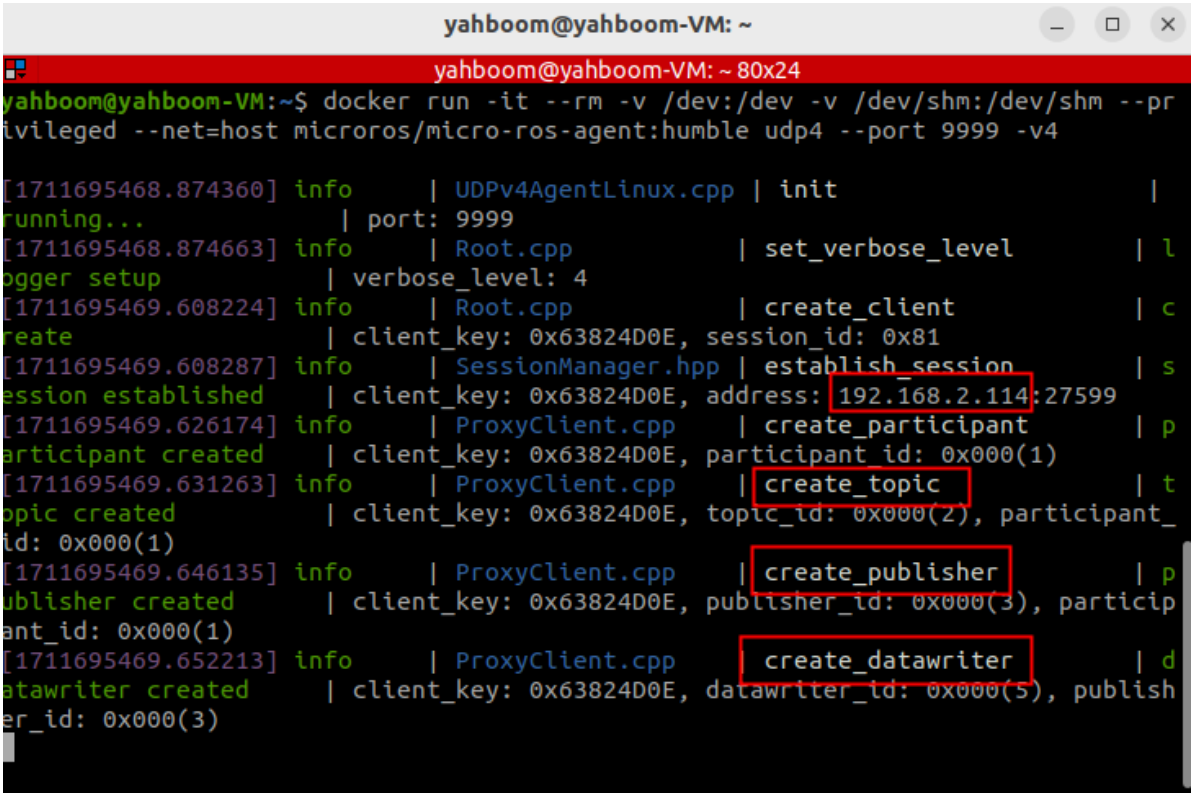
5. 在完成ROS-Wifi图传模块的代理配置完成后，在代理主机执行以下的命令

```
#如果使用的资料的虚拟机直接运行命令
sh start_Camera_computer.sh
```

```
#不是资料提供的虚拟机
docker run -it --rm -v /dev:/dev -v /dev/shm:/dev/shm --privileged --net=host
microros/micro-ros-agent:humble udp4 --port 9999 -v4
```

- 这个9999的端口会被使用进入代理，在个人后续开发的 要避免使用此端口

成功进入的结果如图所示:



一定要有这么多信息才算正确的开启了代理，缺少一个可能需要手动把ROS-Wifi图传模块断电复位
由上图代理结果的信息可以得到ROS-Wifi图传模块的ip:192.168.2.114

6. 查询ROS-Wifi图传模块发布的ROS2结点、话题信息、话题发布频率、话题消息类型

发布节点

```
yahboom@yahboom-VM:~$ ros2 node list
/espRos/Esp32Node
```

话题信息

```
yahboom@yahboom-VM:~$ ros2 topic list
/espRos/esp32camera
/parameter_events
/rosout
yahboom@yahboom-VM:~$
```

```
yahboom@yahboom-VM:~$ ros2 topic echo /espRos/esp32camera
header:
  stamp:
    sec: 1410
    nanosec: 165194
  frame_id: esp32_Camera
format: jpeg
data:
- 255
- 216
- 255
- 224
- 0
- 16
- 74
- 70
- 73
```

消息类型

```
yahboom@yahboom-VM:~$ ros2 topic info /espRos/esp32camera
Type: sensor_msgs/msg/CompressedImage
Publisher count: 1
Subscription count: 0
```

话题发布频率

```
yahboom@yahboom-VM:~$ ros2 topic hz /espRos/esp32camera
average rate: 5.173
        min: 0.128s max: 0.377s std dev: 0.08695s window: 6
average rate: 5.896
        min: 0.117s max: 0.377s std dev: 0.06701s window: 13
average rate: 6.457
        min: 0.111s max: 0.377s std dev: 0.05652s window: 21
average rate: 6.582
        min: 0.111s max: 0.377s std dev: 0.05274s window: 28
average rate: 6.780
        min: 0.111s max: 0.377s std dev: 0.04775s window: 36
average rate: 6.918
        min: 0.111s max: 0.377s std dev: 0.04374s window: 44
average rate: 6.963
        min: 0.111s max: 0.377s std dev: 0.04227s window: 52
average rate: 7.070
        min: 0.111s max: 0.377s std dev: 0.03980s window: 60
average rate: 7.160
        min: 0.108s max: 0.377s std dev: 0.03802s window: 68
average rate: 7.213
        min: 0.108s max: 0.377s std dev: 0.03628s window: 76
average rate: 7.268
        min: 0.108s max: 0.377s std dev: 0.03495s window: 84
```

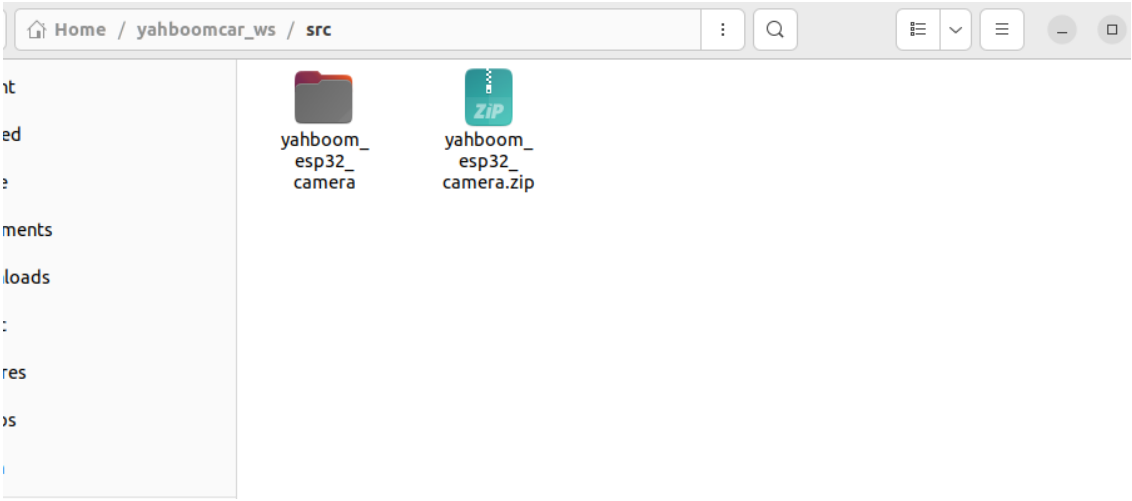
2.ROS2获取摄像头画面

注意：如果使用的是资料提供的虚拟机只看第4步即可

1. 先创建文件夹，执行以下命令

```
mkdir -p ~/yahboomcar_ws/src
```

2. 把资料的提供的**yahboom_esp32_camera**放到刚才创建好的路径里面，并解压



3. 然后功能包编译并添加环境

```
colcon build
echo "source ~/yahboomcar_ws/install/setup.bash " >> ~/.bashrc
source ~/.bashrc
```

4. 运行命令打开摄像头画面

```
ros2 run yahboom_esp32_camera sub_img
```

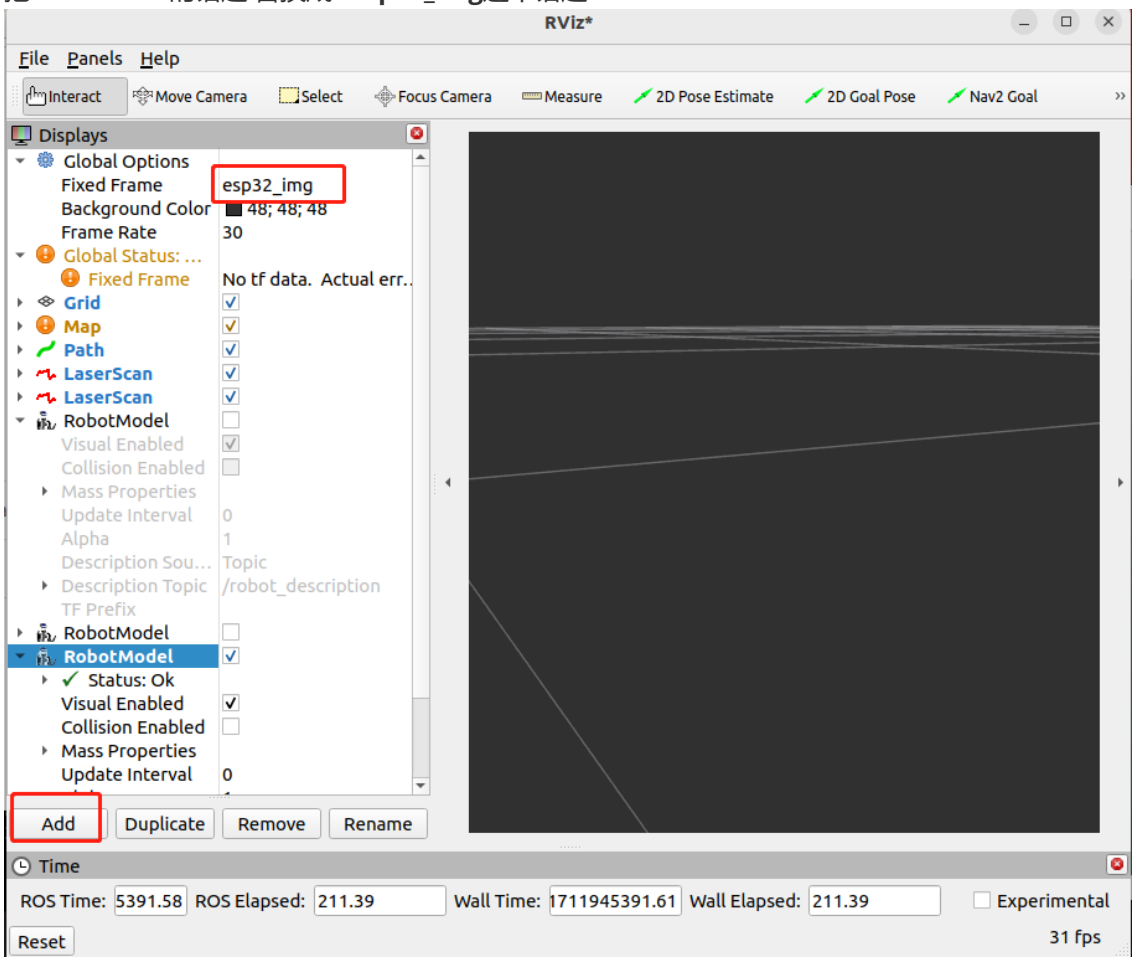


3.在RVIZ里面查看图片

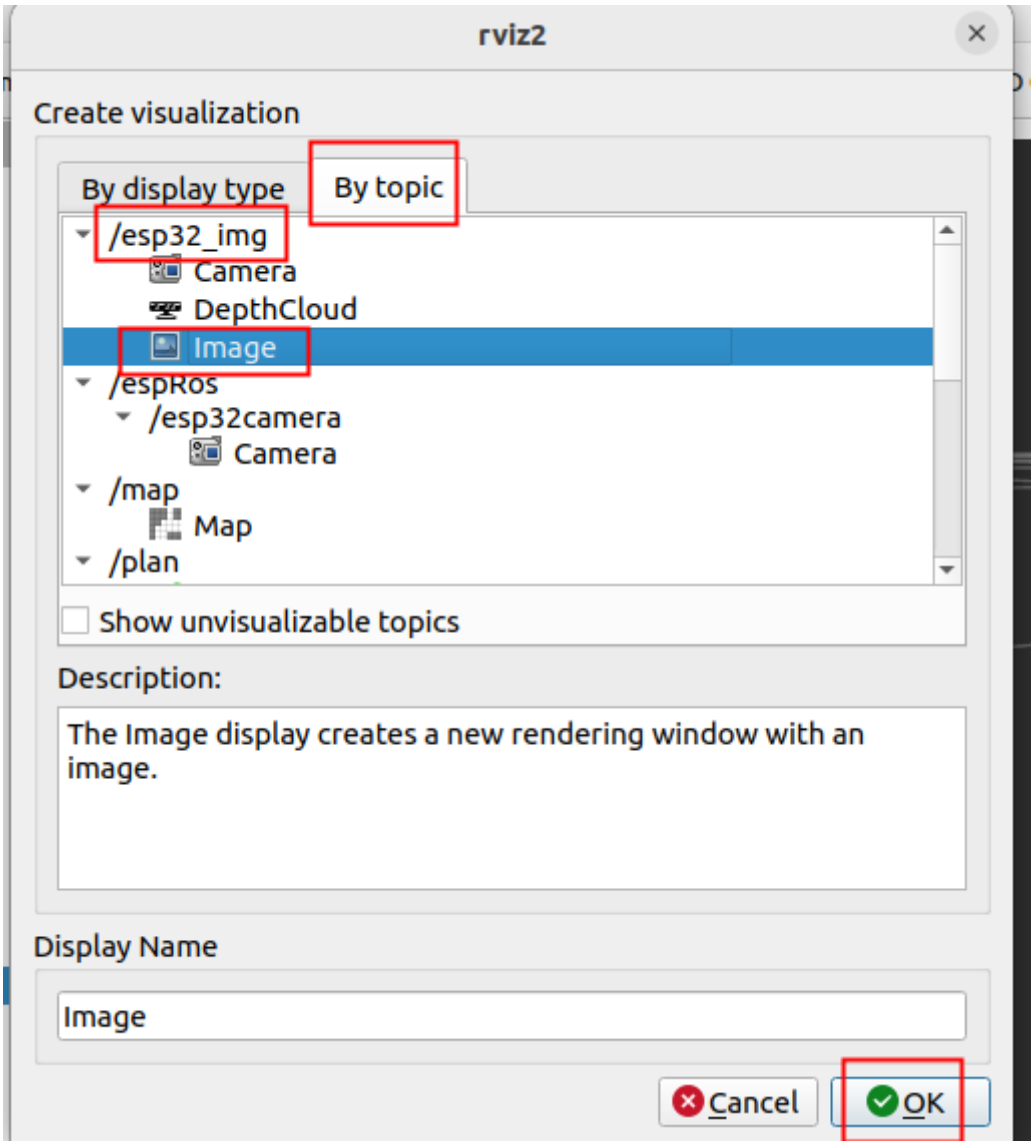
1. 前提条件ROS2获取摄像头画面的条件下，打开安装好的rviz软件

```
rviz2 #rviz/rviz2 资料的虚拟机打开rviz的命令
```

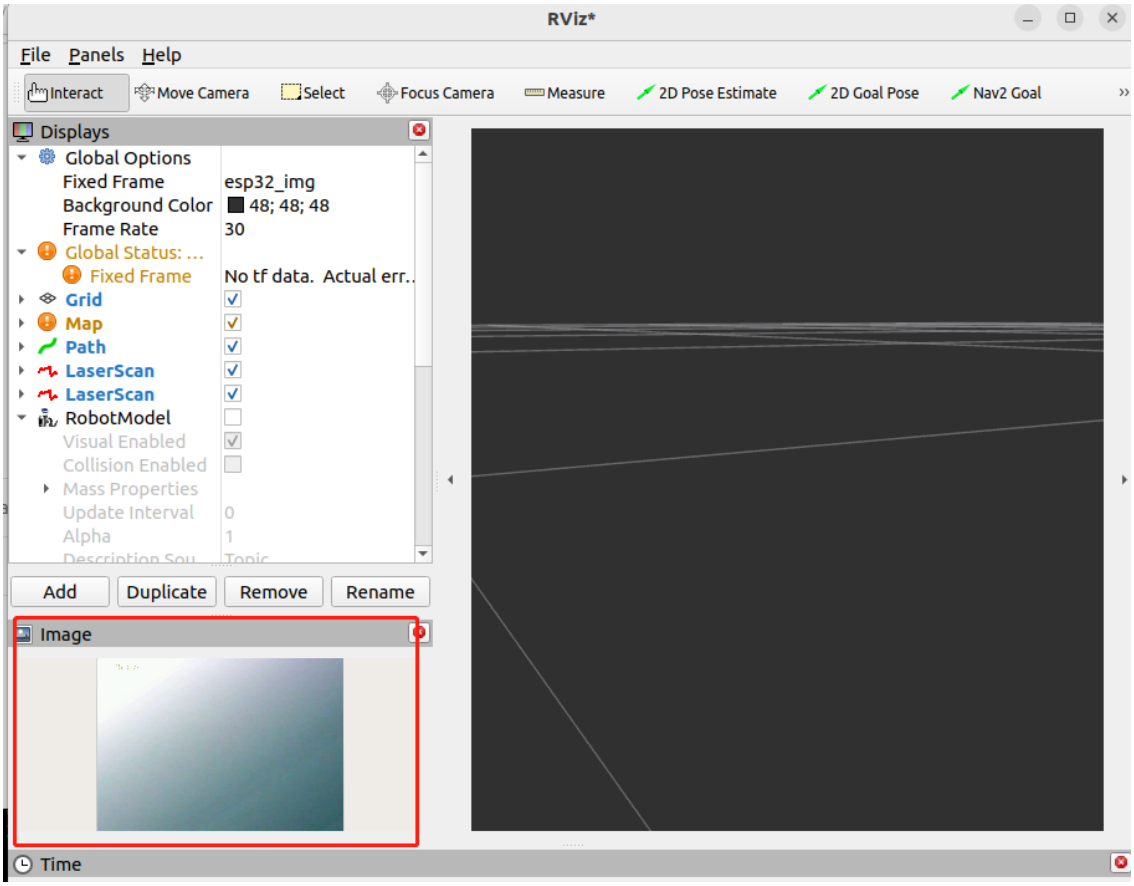
2. 把fixed frame的话题 替换成 **/esp32_img**这个话题



3. 然后加载这话题的图片



4. 最后rviz显示出摄像头的画面



4.注意事项

1. 如果主机主动断开代理，需要把ROS-Wifi图传模块手动断电重启，才能重新恢复，连接代理
2. 这个9999的端口会是进入代理的，在个人后续开发的 要避免使用此端口
3. 使用代理只能通过STA(连接局域网)的模式进行配置。